The reason for asynchronous code

Imagine you are installing a kitchen. You build the cabinets from DIY kits, but you get to a point at which one of the doors is missing a hinge. You decide you can't move forward until you get a hinge, so you drive down to the store to pick one up. All the time you spend driving to the store is wasted and adds a delay to your task. You are working synchronously, completing one task before moving to the next.

A more efficient approach to this scenario would be to call the store and have them deliver a hinge. That way, you can continue with other tasks while waiting for notification (the ringing of your doorbell) that the delivery task you delegated to the store has been completed. This is how asynchronous programming works.

A web server has a limited number of threads available, and in high-load situations, all of the available threads might be in use. When that happens, the server can't process new requests until the threads are freed up. With synchronous code, many threads may be tied up while they aren't actually doing any work because they're waiting for I/O, such as database calls, to complete. With asynchronous code, when a process is waiting for I/O to complete, its thread is freed up for the server to use for processing other requests. As a result, asynchronous code enables server resources to be used more efficiently, and the server is able to handle more traffic without delays.

The next major issue with the generated code may not be obvious at first glance, but it soon becomes clear when you run the application and navigate to /property-manager/ edit?id=1, as figure 8.12 illustrates. The values appearing in the city select list are the key values, not the city names.